



**An End-to-End Quality Assurance Approach for Web, Mobile,
and API Platforms**

Loyalty+ Store (Real Targets)

Technical Report

By

Hala Ahmad Awad Al Hanaqtah

Supervised By

Dr. Ashraf Al-Smadi

**Submitted in Partial Fulfillment of the requirements for the ICT
Upskilling Program**

10th, February 2026

Table of Contents

Contents	Page
1. Executive Summery.....	1
2. Systems Under Test and Assumptions	1
2.1 System Under Test	1
2.2 Assumptions	2
3. Test Strategy and Scope	2
3.1 In-Scope	3
3.2 Out-of-Scope	3
4. Manual Testing	3
4.1 Test scenarios	4
4.2 Test Coverage	4
4.3 Execution Summery.....	7
5. Defects	8
5.1 Severity Distribution	8
5.2 Top Issues	8
6. Postman API Testing.....	17
6.1 Collection and Environment Variables Approach	18
6.2 Data-Driven Approach (CSV File)	18
6.3 Targeted Endpoints	18
6.4 Automation and Reporting	22
7. K6 Performance Testing	23
7.1 Test Plan and Load Profiles	23
7.2 Smoke Test Observation	24
7.3 Load Test Observation	24
7.4 K6 Results Analysis	25
8. UI Automation	26

8.1 Selenium, TestNG Web Automation	26
8.2 Appium Mobile Automation	28
CI/CD Integration	31
9.1 Setup	31
9.2 Pipeline Trigger	31
9.3 Workflow Automation	31
9.4 Test Execution and Reporting	31
Risks, Limitation, and Recommendation	32
10.1 Risks	32
10.2 Limitation	32
10.3 Recommendation	33
Appendices	34

List of Tables

Table number	Table	Page
Table (1.0)	Test scenarios	3
Table (2.0)	Test cases.	4
Table (3.0)	Bug Report 1	8
Table (4.0)	Bug Report 2	9
Table (5.0)	Bug Report 3	11
Table (6.0)	Bug Report 4	12
Table (7.0)	Bug Report 5	13
Table (8.0)	Bug Report 6	14
Table (9.0)	Bug Report 7	15
Table (10.0)	Bug Report 8	16
Table (11.0)	K6 Results	26

List of Figures

Figure number	Figure	Page
Figure (1.0)	Environment variables.	17
Figure (2.0)	Collection variables	18
Figure (3.0)	CSV Files	18
Figure (4.0)	Postman Collection Runner Results	22
Figure (5.0)	HTML Xtra Report	23
Figure (6. 0)	Smoke Test Results	24
Figure (7.0)	Load Test Results	25
Figure (8.0)	Standard_User Test Cases	27
Figure (9.0)	Problem_User Test Cases	28
Figure (10.0)	Test Output Reports	28
Figure (11.0)	"Confirm" Button Assertion Results	29
Figure (12.0)	Quantity Selection Assertion Results	30
Figure (13.0)	Item Selection Assertion Results	30
Figure (14.0)	Input Validation Handling Assertion Results	30
Figure (50)	CI/CD deploy.yml File Run Result	31

1. Executive Summery

The purpose of this documentation is to present an inclusive overview of the testing approach conducted on the following platforms: the SauceDemo web platform, the HyperMax mobile application, and the DummyJSON REST API. The primary goal of this project is to demonstrate the full testing lifecycle, covering functional validation, UI automation, API testing, performance evaluation, and CI/CD integration. This work further aims to validate system correctness, reliability, and stability using the appropriate tools and frameworks by transitioning from manual exploratory sessions to advanced automated frameworks and performance modelling.

The testing process successfully established a baseline for system health while identifying critical logic gaps. Beyond simple verification, the project focused on creating a maintainable quality framework through reusable test strategies and continuous integration practices. The results emphasize the importance of multi-layered validation in detecting role-specific defects and scalability bottlenecks, ultimately providing a clear roadmap for system optimization and industry-ready deployment.

2. Systems Under Test and Assumptions

This section defines the operational boundaries of the project and the conditions where testing was employed to ensure results are consistent and accurate, as shown below:

2.1 System Under Test:

- a. SauceDemo Website (<https://www.saucedemo.com/>), a website used to simulate a real-world online shopping system, which is needed to practice software testing concepts that span from authentication to a complete checkout process.
- b. HyperMax Mobile Application (APK v25.8.2), an e-commerce platform that serves as a target for testing UI interactions, product management, and reliable purchase flow on handheld devices.
- c. DummyJSON API (<https://dummyjson.com/>), a publicly available API that generates realistic data for use in data validation and server-side performance for multiple endpoints, eliminating the need to set up servers.

2.2 Assumptions:

- a. Testing is strict for fake data; no real-world data was used during test execution.
- b. All used frameworks and tools (Selenium/TestNG, Postman, K6, Appium) are assumed to be configured and installed correctly.
- c. The local environment (Windows 11) and Android Emulator (Pixel 5) are stable and maintain consistent internet connectivity.
- d. For API tests, it is assumed that the “access_token” remains valid for the Newman report execution.
- e. The local GitHub Self-Hosted Runner and Appium service are active during execution.

3. Test Strategy and Scope

A multi-layered testing strategy was implemented to maximize coverage and impact:

3.1 In-Scope:

- a. Manual exploratory testing, performed firstly on SauceDemo to understand the platform's overall functionalities, edge cases, and its behavior across all user roles.
- b. A detailed test plan was created to present a clear overview of the testing performed on SauceDemo, covering both manual and automated testing, and defining the testing strategy, scope, environment, roles and responsibilities, risks, entry and exit criteria, and testing timeline.
- c. Manual functional testing's core workflows (authentication, product browsing, filtering, cart management, checkout, and error handling) were validated across all user roles.
- d. For UI automation, test suites were developed on SauceDemo to automate both happy and negative paths based on specific user roles, each in a separate package.
- e. Mobile automation involves certain UI interactions, including country selection, native button clicks, and product quantity specification, which were validated on the HyperMax app.
- f. API testing, CRUD operations were performed on DummyJSON backend functionalities, thus taking four endpoints and two configurations of an environment-based approach and a data-driven approach.
- g. Performance testing employed Smoke and Load testing on specific response time for (P90), (P95), and error rate benchmarks.

3.2 Out-of-Scope:

- a. Testing was restricted to the checkout logic, yet third-party payment providers such as PayPal were excluded.
- b. Cross-Platform compatibility, testing was conducted only on Google Chrome for SauceDemo and the Pixel 5 Emulator for HyperMax.
- c. For UI automation, other roles such as visual, error, locked out, performance glitch user, etc., were left out.
- d. Stress testing was not performed beyond the identified breaking point.

4. Manual Testing

It is the primary method for uncovering crucial logical flaws and inconsistencies before moving to automation.

4.1 Test scenarios

Test scenarios were designed to comprehend the behavior of SauceDemo, including authentication, inventory and sorting of items, cart and checkout cycle, along with negative testing as shown in Table 1.

Table 1: Test scenarios.

ID	Feature	Scenario Description	Type
TS-01	Login	Login with valid username = “standard_user” and password = “secret_sauce”	Happy
TS-02	Login	Login with invalid username= “TestUser” and password= “secret_sauce”	Negative
TS-03	Item Browsing	View Item list after successful login	Happy
TS-04	View Item Description	Displaying another item’s description instead of one that is clicked	Negative
TS-05	Item Sorting	Click on Sort Items by price (low to high)	Happy
TS-06	Cart	Click on Add to Cart button, but the item is not added	Negative
TS-07	Cart	Click on Add Item to cart, and item is added	Happy
TS-08	Cart	Verify cart counter updates correctly after adding item	Happy
TS-09	Cart	Click on Remove Item from cart, and item is removed	Happy

TS-10	Cart	Attempt to remove any item that is not in cart	Negative
TS-11	Checkout	Complete checkout with valid user information	Happy
TS-12	Checkout	Attempt checkout with missing user information	Negative
TS-13	Checkout	Attempt to go directly to confirmation page URL	Negative
TS-14	Checkout	Proceed to check out with an empty cart	Negative
TS-15	Logout	Logout from the application successfully	Happy
TS-16	Main Page Navigation	Attempt to access main page without being logged in	Negative

4.2 Test Coverage

The manual testing suite consisted of 30 inclusive test cases, shown in Table 2, documented in a separate Excel file (ProjectTestCases.xlsx), ensuring full functional coverage of core business flow. Beyond the “standard_user” flow, extensive role-based testing was performed using system-defined users such as “performance_glitch_user,” “error_user,” “visual_user,” “locke_out_user,” and “problem_user” to identify UI inconsistencies and behavioral failures. All results were backed with visual proof captured with ShareX and referenced through external links within the report to ensure transparency and traceability.

Table 2: Test cases.

Test Case ID	Description	Test Steps	Test Data	Expected Result	Actual Result	Pass/Fail	Evidences	Notes
1	Verify Successful Login with Valid Credentials.	1- Open SauceDemo website. 2- Enter "standard_user" in Username field. 3- Enter "secret_sauce" in Password field. 4- Click the "Login" button.	UserName: standard_user, Password: secret_sauce.	The user should be successfully logged in immediately and redirected to the (Products) page.	As Expected.	Pass	https://2cm.es/1mPKD	
2	Verify Adding Items to Cart.	1- Navigate to (Products) page after logging in. 2- Select random item from the list of products. 3- Click "Add to Cart" button for selected item. 4- Observe the cart icon number.	Item Name: Sauce Labs Bike Light.	The item should be added to the cart and the cart icon should display the updated count number of items added.	As Expected.	Pass	https://2cm.es/1hS6V	Log In Data: UserName: standard_user Password: secret_sauce Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Backpack 3. Sauce Labs Bolt T-Shirt 4. Sauce Labs Fleece Jacket 5. Sauce Labs Onesie
3	Verify Filtering and Sorting Items.	1- Navigate to the (Products) page. 2- Click the "Sort/Filter" button. 3- Select the desired sorting option. 4- Observe the order of displayed products.	Filter Type: Price (low to high).	Items of the lowest price should be displayed first in order.	As Expected.	Pass	https://2cm.es/1mPLi	Log In Data: UserName: standard_user Password: secret_sauce Applicable Items (same behavior): 1. Name (A to Z) 2. Name (Z to A) 3. Price (high to low)
4	Verify Removing Items from Cart.	1- After adding items to cart. 2- Click the "Remove" button below selected item. 3- Observe the cart icon number after removing an item.	Item Name: Sauce Labs Backpack.	The item should be removed from the cart successfully, and the cart icon count number should decrease by one for each removed item.	As Expected.	Pass	https://2cm.es/1mPLr	Log In Data: UserName: standard_user Password: secret_sauce Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Bike Light 3. Sauce Labs Bolt T-Shirt 4. Sauce Labs Fleece Jacket 5. Sauce Labs Onesie

5	Verify Checkout - Filling in Required User Information.	1- Navigate to (Your Cart) page with items added. 2- Click the "Checkout" button. 3- On the (Checkout: Your Information) page, fill in the required fields. 4- Click the "Continue" button.	First Name: Hala, Last Name: Hanagtah, Zip Code: 11196.	The system should accept the entered information and navigate to the (Checkout: Overview) page displaying selected items information and payment, shipping information as well as total price.	As Expected.	Pass	https://2cm.es/1hS8S	Log In Data: UserName: standard_user Password: secret_sauce
6	Placing an Order.	1- After filling in the required information and clicking on "Continue". 2- On the (Checkout: Overview) page, click the "Finish" button. 3- Verify the navigation to (Checkout: Complete) page.	1- Items added to cart. 2- User information entered previously. 3- Clicking on "Finish Button".	The system should place the order successfully and redirect user to (Checkout: Complete) page and display: "Thank you for your order! Your order has been dispatched, and will arrive just as fast as the pony can get there!" message.	As Expected.	Pass	https://2cm.es/1hS8c	Log In Data: UserName: standard_user Password: secret_sauce
7	Verify Viewing Item Description.	1- Navigate to the (Products) page. 2- Click on the name of an item. 3- Verify that a new page opens displaying the item's information.	Item Name: Sauce Labs Onesie.	The system should navigate to a new page displaying the correct item name, description, price, image, and "Add to Cart" button.	As Expected.	Pass	https://2cm.es/1mPMG	Log In Data: UserName: standard_user Password: secret_sauce Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Bike Light 3. Sauce Labs Bolt T-Shirt 4. Sauce Labs Fleece Jacket 5. Sauce Labs Backpack
8	Verify Error Message When Required Fields Are Empty.	1- Navigate to Checkout: (Your Information) page. 2- Leave all required fields empty. 3- Click the "Continue" button.	First Name: Last Name: Zip Code:	A corresponding error message should be displayed for each empty field, indicating: "Error: First Name is required", and navigation to next page is prevented.	As Expected.	Pass	https://2cm.es/1hS8t	Log In Data: UserName: standard_user Password: secret_sauce Same Behavior for: 1. Last Name Field -> "Error: Last Name is required" 2. Postal Code Field -> "Error: Postal Code is required"

9	Verify Checkout with Empty Cart.	1- Navigate to (Products) page and do not add items to the cart. 2- Click on the Cart icon to open (Your Cart) page then click on "Checkout" button. 3- On (Checkout: Your Information) page, fill in the required fields then click on "Continue" button. 4- On (Checkout: Overview) page, click on "Finish" button.	First Name: Hala Last Name: Ahmad Postal Code: 1113	The system should prevent checkout when the cart contains zero items, by displaying an error message as "Your cart is empty. Add items before checking out" and prevent the user navigating to (Checkout Complete) page.	The system allowed checkout to complete with an empty cart and navigated successfully to the (Checkout: Complete) - Order Confirmation) page	Fail	https://2cm.es/1mPMT	Log In Data: UserName: standard_user Password: secret_sauce
10	Verify Logout Process.	1- Click the "menu" button on (Products) page. 2- Select "Logout" from the menu options. 3- Verify that the user is logged out. 4- Verify that the user is redirected to (Login) page.	1- User logged in with valid credentials. 2- Clicking on "Menu" button then Clicking on "Logout" option.	The user should be logout from the system successfully then redirect the user to (Login) page.	As Expected.	Pass	https://2cm.es/1mPN1	Log In Data: UserName: standard_user Password: secret_sauce
11	Verify Login Attempt with Locked User.	1- Open the SauceDemo website. 2- Enter "locked_out_user" in Username field. 3- Enter "secret_sauce" in Password field. 4- Click the "login" button. 5- Observe any error messages.	UserName: locked_out_user, Password: secret_sauce.	User cannot log in with locked-out credentials, and the system displays error message: "Epic sadface: Sorry, this user has been locked out."	User is unable to log in. System displays error message: "Epic sadface: Sorry, this user has been locked out."	Pass	https://2cm.es/1mPNo	Log In Data: UserName: locked_out_user Password: secret_sauce
12	Verify Correct Items' Images.	1- Navigate to (Products) page after logging in. 2- Verify that the correct image is displayed for each item.	The Image of Sauce Labs Bolt T-Shirt.	The System should display the correct corresponding image for Sauce Labs Bolt T-Shirt along with its name, description, and price.	The System displays an image of a dog holding a ball in his mouth instead of the correct item's image.	Fail	https://2cm.es/1hS99	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Bike Light 3. Sauce Labs Onesie 4. Sauce Labs Fleece Jacket 5. Sauce Labs Backpack

13	Verify Adding Items to Cart (1).	1- Navigate to (Products) page after logging in. 2- Click "Add to Cart" for specific items. 3- Observe the cart icon count after each addition.	Item Name: Sauce Labs Backpack	The items should be added to the cart and the cart icon should display the updated number of items added.	As Expected.	Passed	https://2cm.es/1hS9s	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Sauce Labs Onesie 2. Sauce Labs Bike Light
14	Verify Adding Items to Cart (2).	1- Navigate to (Products) page after logging in. 2- Click "Add to Cart" for specific items. 3- Observe the cart icon count after each addition.	Item Name: Sauce Labs Fleece Jacket.	The items should be added to the cart and the cart icon should display the updated number of items added.	The item were not added to the cart, and the cart icon count remained unchanged.	Fail	https://2cm.es/1hS9s	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Bolt T-Shirt
15	Verify Removing Items from Cart.	1- Navigate to (Products) page after logging in. 2- After adding selected items. 3- Click on "Remove" button for added items from (Products) page. 4- Observe the cart icon count after each removal.	Item Name: Sauce Labs Bike Light.	Each item clicked for Remove should be removed from the cart, and the cart icon count should decrease by one per removed item.	The items were not removed from the cart, and the cart icon count remains the same.	Fail	https://2cm.es/1mPMT	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Sauce Labs Backpack. 2. Sauce Labs Onesie
16	Verify Filtering and Sorting Items.	1- Navigate to the (Products) page. 2- Click the "Sort/Filter" button. 3- Select the desired sorting option. 4- Observe the order of displayed products.	Filter Type: Price (high to low).	Items of the highest price should be displayed first in order.	The order of the itmes did not change (remained the same)	Fail	https://2cm.es/1hS9T	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Name (A to Z) 2. Name (Z to A) 3. Price(low to high)
17	Verify Correct Item Description.	1- Navigate to (Products) page. 2- Click on the name of any item. 3- Verify that a new page opens displaying the item's correct information.	Item Name: Test.allTheThings() T-Shirt (Red).	The System should navigate to a new page displaying the correct item name, description, price, image, and "Add to Cart" button.	The system navigates to a new page, but it displays information of Sauce Labs Backpack instead of the selected item.	Fail	https://2cm.es/1mPOM	Log In Data: UserName: problem_user Password: secret_sauce Applicable Items (same behavior): 1. Sauce Labs Bolt T-Shirt. 2. Sauce Labs Backpack. 3. Sauce Labs Bike Light. 4. Sauce Labs Onesie.
18	Verify Checkout – Last Name Field Validation.	1- Add random items to the cart. 2- Click the "Checkout" button on (Your Cart) page. 3- On the (Checkout: Your Information) page, enter required information. 4- Click "Continue" button to proceed.	First Name: Shahed, Last Name: Aref, Zip Code: 11280.	The user should be able to enter all fields, and then navigates to the (Checkout:Overview) step.	User can't enter a value in Last Name field; It's is redirected to First Name field. Clicking Continue displays a "Last Name is required" error message.	Fail	https://2cm.es/1mPOG	Log In Data: UserName: problem_user Password: secret_sauce
19	Verify Reset App State Functionality.	1- Log in and add random items to the cart. 2- Click the "Menu" button. 3- Select Reset App State. 4- Observe the cart contents, cart count, and (Products) page buttons.	1- Valid logged in user. 2- Add Sauce Labs Backpack and Sauce Labs Bike Light to cart. 3- Clicking on Reset App Status Button.	All items should be removed from the cart, the cart count reset to zero, and the (Products page) should show all "Add to Cart" buttons in their original state.	All items were removed from the cart, and the cart count was reset to zero, but the (Products page) buttons remained in Remove state, as if the items were still in the cart.	Fail	https://2cm.es/1mPOV	Log In Data: UserName: problem_user Password: secret_sauce
20	Verify Viewing Item number 4 Description.	1- Navigate to (Products) page. 2- Click on specific item to view its description. 3- Verify that a new page displays the item's correct information.	Item Name: Sauce Labs Fleece Jacket.	The System should display the item's details page and correct title, description, and price corresponding to Sauce Labs Fleece Jacket.	The System displays the title as "Item not found", description shows an error message (e.g., "We're sorry, but your call could not be completed. Please try again."), and price is not displayed correctly.	Fail	https://2cm.es/1hSaF	Log In Data: UserName: problem_user Password: secret_sauce
21	Verify Cart Icon Position on (Products) Page.	1- Navigate to (Products) page after logging in. 2- Observe the position of the Cart icon relative to the "Filtering" button.	1- UserName: visual_user, Password: secret_sauce. 2- (Products) page. 3- Cart icon position.	The Cart icon should be displayed at the top-right corner of the (Products) page, positioned to the right of the "Filtering" button.	The Cart icon is displayed at the top of the page, to the left of the "Filtering" button.	Fail	https://2cm.es/1mPQc	Login Data: UserName: visual_user, Password: secret_sauce.
22	Verify Cart Icon Visibility on different pages.	Verify the Cart icon visibility on the following pages: 1- Your Cart 2- Checkout Overview 3- Checkout Complete 4- Checkout Your Information	1- (Your Cart, Checkout: Overview, Checkout: Complete, Checkout: Your information) page. 2- Cart icon visibility.	The Cart icon should only be visible on the Products page and It should not appear on Checkout Overview, Checkout Complete, or Checkout Your Information pages.	The Cart icon is displayed on all pages.	Fail	https://2cm.es/1mPQc	Login Data: UserName: visual_user, Password: secret_sauce.

23	Verify Checkout Button Position.	1- After valid Login. 2- Add Items to cart and navigate to (Your Cart) page. 3- Observe the position of the "Checkout" button.	1- Added Sauce Labs Fleece Jacket. 2- Cart Icon Button Position.	It should be positioned at the bottom-right of the (Your Cart) page.	It is displayed at the top-right of the (Your Cart) page.	Fail	https://2cm.es/1h5c8	Login Data: UserName: visual_user, Password: secret_sauce.
24	Verify Filtering and Sorting Items.	1- Navigate to (Products) page. 2- Click the "Sort/Filter" button. 3- Select the desired sorting option. 4- Observe the order of displayed products.	Filter Type: Name (A to Z).	All items should be displayed in ascending alphabetical order.	The products are not sorted as expected, and a pop-up dialog box is displayed with the message: "Sorting is broken. This error has been reported to Backtrace."	Fail	https://2cm.es/1mPQl	Login Data: UserName: error_user, Password: secret_sauce. Applicable Items (same behavior): 1. Name (Z to A) 2. Price(high to low) 3. Price(low to high)
25	Verify Checkout – Last Name Field Validation.	1- Add random items to the cart. 2- Click the "Checkout" button on the Cart page. 3- On (Checkout: Your Information) page, enter required information. 4- Click "Continue" button to proceed.	First Name: Noha, Last Name: Mansour, Zip Code: 10211.	The System should allow entry of First Name, Last Name, and Zip Code, and if any field is missing, it should display a validation error message "Error: Last Name is required" and prevent navigation to (Checkout: Complete!).	The Last Name field does not accept input, and clicking "Continue" button navigates to (Checkout: Complete!) without displaying "Error: Last Name is required" validation error.	Fail	https://2cm.es/1mPQT	Login Data: UserName: error_user, Password: secret_sauce.
26	Verify Order Placement and Checkout Completion on Finish Action.	1- Fill in the required information on the (Checkout:Your Information) page and click "Continue" button. 2- On the (Checkout: Overview) page, click the "Finish" button. 3- Verify the navigation to (Checkout: Complete!) page.	1- User information entered previously. 2- Items added to cart as Sauce Labs Fleece Jacket and Sauce Labs Backpack 3- Clicking on "Finish Button".	The System should successfully place the order and redirects user to "Checkout: Complete" page, displaying the confirmation message: "Thank you for your order! Your order has been dispatched, and will arrive just as fast as the pony can get there!"	There's no action performed. The order is not placed, and the user is not redirected to "Checkout: Complete" page or shown the order confirmation message.	Fail	https://2cm.es/1h5cB	Login Data: UserName: error_user, Password: secret_sauce.
27	Verify Login-In Performance.	1- Open the SauceDemo website. 2- Enter username "performance_glitch_user". 3- Enter password secret_sauce. 4- Click the "Login" button. 4- Observe the time taken to redirect to (Products) page.	UserName: performance_glitch_user, Password: secret_sauce.	The user should be logged in immediately and redirected to the (Products) page without noticeable delay.	After clicking "Login" button, the System waits approximately 2–3 seconds before redirecting the user to the (Products) page.	Fail	https://2cm.es/1h5cZ	Login Data: UserName: performance_glitch_user, Password: secret_sauce.
28	Verify Filtering and Sorting Items Performance.	1- Navigate to the (Products) page. 2- Click the "Sort/Filter" button. 3- Select the desired sorting option. 4- Observe the time taken for the filtered items to be displayed.	Filter Type: Name (Z to A).	The items should be displayed immediately in descending alphabetical order, without any noticeable delay.	The System takes approximately (2–3) seconds before displaying the new order of filtered items.	Fail	https://2cm.es/1h5cR	Login Data: UserName: performance_glitch_user, Password: secret_sauce. Applicable Items (same behavior): 1. Name (A to Z) 2. Price(high to low) 3. Price(low to high)
29	Verify "Back to Products" Navigation Performance.	1- Navigate to the (Products) page after login. 2- Click on any item name to open the item description page. 3- Click the "Back to products" button. 4- Observe the time taken to navigate back to the (Products) page.	Item Name: Sauce Labs Backpack.	The System should navigate back to the Products page instantly with normal response time and without delay.	The System waits approximately (2–3) seconds before navigating back to (Products) page.	Fail	https://2cm.es/1mPqS	Login Data: UserName: performance_glitch_user, Password: secret_sauce.
30	Verify Item's Price Stability.	1- Verify successful login and navigation to the Products page. 2- Click on "Menu" button. 3- Click on "All Items" option. 4- Observe the item list and prices displayed on (Products) page. 5- Click "All Items" option again. 6- Observe the item prices.	Item Name: Sauce Labs Backpack.	Clicking on "All Items" option should consistently display the same price (\$29.99) for Sauce Labs Backpack.	Each time the "All Items" option is clicked, it display a different price each time for Sauce Labs Backpack.	Fail	https://2cm.es/1h5eZ	Login Data: UserName: visual_user, Password: secret_sauce. Applicable Items (same behavior): 1. Test.allTheThings() T-Shirt (Red) 2. Sauce Labs Bike Light 3. Sauce Labs Onesie 4. Sauce Labs Fleece Jacket 5. Sauce Labs Bolt T-Shirt

4.3 Execution Summary

The manual testing phase produced 30 documented test cases, yielding 11 passes and 19 failures. This elevation of failures was intentional, driven by role-based testing to expose system defects across the UI layer, such as incorrect product images or descriptions, and the operational layer, such as items that cannot be added or removed from the cart or a

complete checkout process with an empty cart, representing a significant logical flow. A total of nine defects were reported, ranging from basic to severe logic failure. Whereas all of these issues would go undetected in the standard happy scenario, which confirmed a stable baseline for primary functionality. Overall, the execution results demonstrate that SauceDemo lacks robustness when faced with non-standard scenarios.

5. Defects

To support business-critical impact, comprehensive defect coverage has been provided through 9 detailed bug reports that went beyond simple “Pass/Fail” status, expected and actual results, environment used, descriptions to also include severity and priority assessments supported by screenshots of failures.

5.1 Severity Distribution

The identified bugs were categorized based on their impact on the functionalities and user experience.

- a. High severity, where issue blocks the essential features of the system or bypass system’s validation.
- b. Medium severity, where UI malfunction is involved disrupting the overall experience.
- c. Low severity, where minor issues take place with no effect on functionalities or experience.

5.2 Top Issues

Samples of top issues found are illustrated in (Table 3-10).

Table 3: Bug Report 1

Category	Label	Value
Bug ID	ID	9
	Name	Verify Checkout with Empty Cart.
	Reporter	Hala Hanaqtah.
	Submit Date	1/15/2026, 4:15PM

Bug Overview	Summery	The system allows users to complete checkout even when the cart is empty, when logged in as standard_user.
	ScreenShot	https://2cm.es/1mPMT
	URL	https://www.saucedemo.com/cart.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Details	Description	When a user attempts to checkout with zero items in the cart, the system does not block the process. After filling in required information and clicking "Finish" button, the user is incorrectly navigated to the (Checkout: Complete!) page.
	Steps to Reproduce	1- Open the SauceDemo website. 2- Enter "standard_user" in Username field. 3- Enter "secret_sauce" in Password field. 4- Navigate to (Products) page and do not add items to the cart. 5- Click on Cart Icon to open (Your Cart) page then click on "Checkout" button. 6- On (Checkout: Your Information) page, fill in the required fields then click on "Continue" button. 7- On (Checkout: Overview) page, click on "Finish" button.
	Expected Result	The system should prevent checkout when the cart contains zero items, by displaying an error message as "Your cart is empty. Add items before checking out" and prevent the user navigating to (Checkout Complete) page.
	Actual Result	The system allowed checkout to complete with an empty cart and navigated successfully to the (Checkout: Complete!) - Order Confirmation) page.
Bug Tracking	Severity	High
	Priority	High

Table 4: Bug Report 2

Category	Label	Value
Bug ID	ID	18

	Name	Verify Checkout – Last Name Field Validation.
	Reporter	Hala Hanaqtah.
	Submit Date	1/15/2026, 4:35PM
Bug Overview	Summery	User is unable to enter a value in the Last Name field during checkout, preventing navigation to the next step when logged in as problem_user.
	ScreenShot	https://2cm.es/1mPOG
	URL	https://www.saucedemo.com/checkout-step-one.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	During checkout, after adding items to cart and proceeding to (Checkout: Your Information) page, the Last Name field does not accept input, and focus is redirected to First Name field. Clicking "Continue" triggers a “Last Name is required” error, blocking progression to (Checkout: Overview) page. This prevents completing the checkout process.
	Steps to Reproduce	1- Open the SauceDemo website. 2- Enter "problem_user" in Username field. 3- Enter "secret_sauce" in Password field. 4- Navigate to (Products) page. 5- Add random items to the cart. 6- Click the "Checkout" button on (Your Cart) page. 7- On the (Checkout: Your Information) page, enter required information. 8- Click "Continue" button to proceed.
	Expected Result	The user should be able to enter all fields and then navigate to the (Checkout:Overview) page.
	Actual Result	Users can't enter a value in Last Name field; It's redirected to First Name field. Clicking "Continue" button displays a “Last Name is required” error message.
Bug Tracking	Severity	High.
	Priority	High.

Table 5: Bug Report 3

Category	Label	Value
Bug ID	ID	14
	Name	Verify Checkout – Last Name Field Validation.
	Name	Verify Adding Items to Cart (2).
	Submit Date	1/15/2026, 4:35PM
Bug Overview	Summery	Certain items cannot be added to cart when logged in as “problem_user”.
	Screenshot	https://2cm.es/1hS9s
	URL	https://www.saucedemo.com/inventory.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	Attempting to add certain items to the cart fails. Clicking the "Add to Cart" button for items such as (Sauce Labs Bolt T-Shirt, Sauce Labs Fleece Jacket, Test.allTheThings() T-Shirt (Red)). The cart icon count does not increase, and the button state does not reflect that the item was added.
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: problem_user Password: secret_sauce 3- On the Products page, try to add these items to the cart: - Sauce Labs Bolt T-Shirt - Sauce Labs Fleece Jacket - Test.allTheThings() T-Shirt (Red) 4- Observe the cart icon and button behavior.
	Expected Result	The items should be added to the cart and the cart icon should display the updated number of items added.
	Actual Result	The item was not added to the cart, and the cart icon count remained unchanged.
Bug Tracking	Severity	High.
	Priority	High.

Table 6: Bug Report 4

Category	Label	Value
Bug ID	ID	24
	Name	Verify Filtering and Sorting Items.
	Reporter	Hala Hanaqtah.
	Submit Date	1/11/2026, 12:30PM
Bug Overview	Summery	Item Sorting and Filtering Malfunction when logging in as error_user.
	Screenshot	https://2cm.es/1mPQI
	URL	https://www.saucedemo.com/inventory.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	Applying the filter/sort function on the Products page fails, it does not sort items in the correct order. A pop-up dialog box appears with the message: "Sorting is broken. This error has been reported to Backtrace."
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: error_user Password: secret_sauce 3- Navigate to the (Products) page. 4- Click on the Sort/Filter button. 5- Select desired option. 6- Observe the order of displayed items and check for any pop-up messages.
	Expected Result	All items should be displayed in the correct order corresponding to the selected option.
	Actual Result	The products are not sorted as expected, and a pop-up dialog box is displayed with the message: "Sorting is broken. This error has been reported to Backtrace."
Bug Tracking	Severity	Medium.
	Priority	Medium.

Table 7: Bug Report 5

Category	Label	Value
Bug ID	ID	12
	Name	Verify Correct Items' Images.
	Reporter	Hala Hanaqtah.
	Submit Date	1/11/2026, 9:35PM
Bug Overview	Summery	It displays an incorrect item image of a dog holding a ball in his mouth instead of the correct item image when logged in as problem_user.
	Screenshot	https://2cm.es/1hS99
	URL	https://www.saucedemo.com/inventory.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	When viewing products on the Products page after logging in, the images displayed for the items do not correspond to their respective items. Instead of showing Sauce Labs Bolt T-Shirt correct image, it displays the same incorrect image of (a dog holding a ball in his mouth.)
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: problem_user Password: secret_sauce 3- After login, view the Products page. 4- Observe the images displayed for Sauce Labs Bolt T-Shirt.
	Expected Result	The System should display the correct corresponding image for Sauce Labs Bolt T-Shirt along with its name, description, and price.
	Actual Result	The System displays an image of (a dog holding a ball in his mouth) instead of the correct item's image.
Bug Tracking	Severity	Medium.
	Priority	Medium.

Table 8: Bug Report 6

Category	Label	Value
Bug ID	ID	25
	Name	Verify Checkout – Last Name Field Validation.
	Reporter	Hala Hanaqtah.
	Submit Date	1/11/2026, 12:00PM
Bug Overview	Summery	Last Name Field Does Not Accept Input and Checkout Validation Fails when logged in as error_user.
	Screenshot	https://2cm.es/1mPQT
	URL	https://www.saucedemo.com/checkout-step-one.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	The (Checkout: Your Information) page fails to properly validate the Last Name field. It accepts input for FirstName, Zip Code fields, but not for LastName field. Then clicking the "Continue" button bypasses validation and navigates directly to (Checkout:Overview) page, without displaying the required validation error message.
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: error_user Password: secret_sauce 3- Add any item to the cart and click "Checkout" button. 4- On (Checkout: Your Information) page: - Enter a value in the First Name field. - Attempt to enter a value in the Last Name field. - Enter a value in the ZIP Code field. - Click the "Continue" button.
	Expected Result	The System should allow entry of First Name, Last Name, and Zip Code, and if any field is missing, it should

		display a validation error message "Error: Last Name is required" and prevent navigation to (Checkout: Complete!).
	Actual Result	The Last Name field does not accept input and clicking "Continue" button navigates to (Checkout: Complete!) without displaying "Error: Last Name is required" validation error.
Bug Tracking	Severity	High.
	Priority	High.

Table 9: Bug Report 7

Category	Label	Value
Bug ID	ID	30
	Name	Verify Item's Price Stability.
	Reporter	Hala Hanaqtah.
	Submit Date	1/15/2026, 3:50PM
Bug Overview	Summery	Item's price on (Products) page changes every time the "All Items" option is clicked when logged in as visual_user.
	Screenshot	https://2cm.es/1hSez
	URL	https://www.saucedemo.com/inventory.html
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	Item prices on the Products page change each time the "All Items" option is clicked from the Menu. Prices should remain consistent, but repeated clicks cause different prices to be displayed for the same items (e.g., Sauce Labs Backpack).
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: visual_user Password: secret_sauce 3- Navigate to (Products) page. 4- Click on "Menu" button. 5- Click on "All Items" option. 6- Observe the item list and prices.

		7- Click "All Items" option again. 8-Observe the item prices.
	Expected Result	Clicking on "All Items" option should consistently display the same price (\$29.99) for Sauce Labs Backpack.
	Actual Result	Each time the "All Items" option is clicked, it displays a different price each time for Sauce Labs Backpack.
Bug Tracking	Severity	Medium.
	Priority	High.

Table 10: Bug Report 8

Category	Label	Value
Bug ID	ID	20
	Name	Verify Viewing Item number 4 Description.
	Reporter	Hala Hanaqtah.
	Submit Date	1/11/2026, 11:15PM
Bug Overview	Summery	Incorrect Item Description for Sauce Labs Fleece Jacket when logged in as problem_user.
	Screenshot	https://2cm.es/1hH21
	URL	https://www.saucedemo.com/inventory-item.html?id=6
Environment	Platform	Web Application.
	Operating System	Windows 11.
	Browser	Google Chrome / Microsoft Edge.
Bug Tracking	Description	Clicking on the Sauce Labs Fleece Jacket on (Products) page, it navigates to description page, but displays incorrect title "Item Not Found", and an error message ("We're sorry, but your call could not be completed. Please try again") as description, and an incorrect price.
	Steps to Reproduce	1- Open SauceDemo website. 2- Log in using the following credentials: Username: problem_user Password: secret_sauce 3- On the Products page, click on Labs Fleece Jacket. 4- Observe the displayed page, title, description, price, and image.

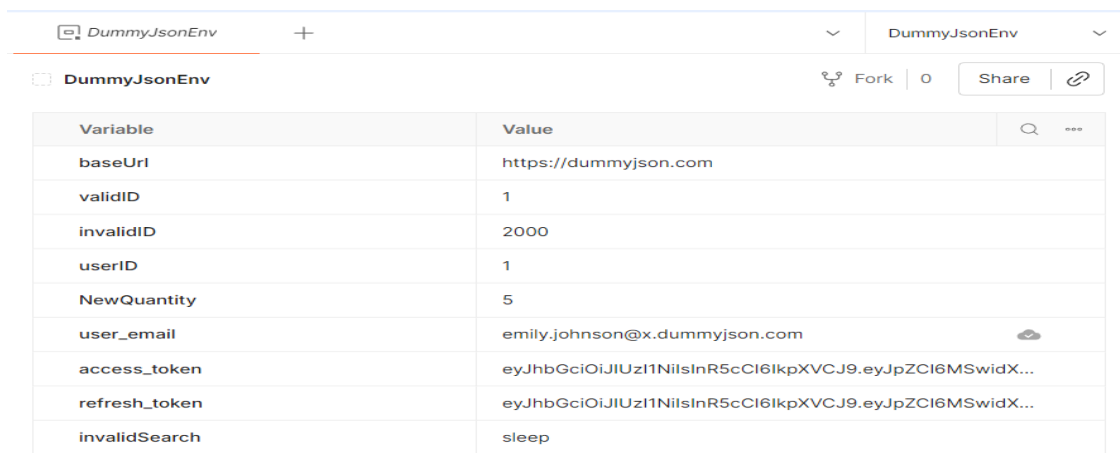
	Expected Result	The System should display the item's details page and correct title, description, and price corresponding to Sauce Labs Fleece Jacket.
	Actual Result	The System displays the title as “Item not found”, description shows an error message (e.g., “We’re sorry, but your call could not be completed. Please try again.”), and price is not displayed correctly.
Bug Tracking	Severity	High.
	Priority	High.

6. Postman API Testing

This section details the automated validation performed on DummyJSON API services, focusing on data integrity, endpoint reliability, and robust error handling through a specific collection design that leverages three configuration layers of:

6.1 Collection and Environment Variables Approach

The purpose is to use dynamic variables across multiple requests, ensuring more flexible and maintainable tests, as shown in Figure 1 and 2. Therefore, data can be implemented directly into request bodies, URLs, headers, and assertions.



The screenshot shows the Postman interface for an environment named 'DummyJsonEnv'. At the top, there's a header with the environment name and a plus icon. Below the header, there's a table with two columns: 'Variable' and 'Value'. The table contains the following data:

Variable	Value
baseUrl	https://dummyjson.com
validID	1
invalidID	2000
userID	1
NewQuantity	5
user_email	emily.johnson@x.dummyjson.com
access_token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidX...
refresh_token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidX...
invalidSearch	sleep

Figure 1: Environment variables

DummyJasonFinalProject

Fork0

Share

Overview

Authorization

Scripts

Variables

Runs

Variable	Value	
searchProduct	iPhone	
NewProductID	195	
NewProductTitle	swarovski swan bracelet	
NewProductPrice	29.99	
updatedTitle	Chicken Nuggets	
searchName	Emily	

Figure 2: Collection variables

6.2 Data-Driven Approach (CSV File)

This approach enables seamless automated testing for multiple users and edge cases simultaneously, where each row represents a unique test scenario, allowing tests to be run with a predefined number of iterations, as shown in Figure 3. Consequently, it eliminates the need to manually modify test requests, making them more scalable, consistent, and easier to maintain.

UserFirstName	UserLastName	UserAge	UserEmail	Search_Name	Invalid_Search	Valid_Id	Invalid_Id
Jhon	Doe	22	Jhon.Doe22@gmail.com	Emily	sleep	1	2000
Demo	User	25	Demo.User25@gmail.com	Emily	jump	2	500
Jane	Smith	20	Jane.Smith25@gmail.com	Emily	zzzz	3	300

Figure 3: CSV File

6.3 Targeted Endpoints

A. Products, Carts, and Authentication requests were validated using Collection and Environment variables (DummyJsonEnv).

Sample Endpoints' Requests:

1. POST /products /Add

- Purpose: Adds a new product to DummyJSON with the specified details.
- Endpoint: POST `{{baseUrl}}/products/add`
- Body: {
" id": `{{NewProductID}}`,

```
"title": "{{NewProductTitle}}",  
"price": {{NewProductPrice}}  
}
```

- Assertions:
 - a. Status should be 201 Created.
 - b. Response should be a JSON object.
 - c. Response should include the keys: id, title, price.
 - d. Response fields should match the input values exactly:
 - id = NewProductID
 - title = NewProductTitle
 - price = NewProductPrice

2. PATCH /carts /id

- Purpose: Updates existing cart's products and/or quantities for a specific user.
- Endpoint: PATCH {{baseUrl}}/carts/{{validID}}
- Body: {
 "userId": {{userID}},
 "products": { [
 {
 "id": 1,
 "quantity": {{NewQuantity}}
 }
] }
}
- Assertions:
 - a. Status should be 200 OK.
 - b. The returned userId should match the userID sent in the request.
 - c. The quantity of the updated product should match the new quantity provided in the request body.

3. POST auth /login New

- Purpose: Authenticates a user with their username and password and returns user details along with access and refresh tokens.
- Endpoint: POST `{{baseUrl}}/auth/login`
- Body: {


```

        "username": "emilys",
        "password": "emilypass"
      
```
- Assertions:
 - a. Status should be 200 OK.
 - b. Response should include the following properties:
 - `id` → User ID
 - `email` → User email
 - `accessToken` → JWT access token
 - `refreshToken` → JWT refresh token
 - c. The following values should be saved in environment variables for future requests:
 - `userID` → User ID
 - `user_email` → User email
 - `access_token` → Access token
 - `refresh_token` → Refresh token

B. Users' endpoint was validated using the CSV file (UsersData.csv).

1. GET `/users /search`
 - Purpose: Searches for users whose first names contain the specified search keyword saved in CSV file.
 - Endpoint: GET `{{baseUrl}}/users/search?q={{Search_Name}}`
 - Assertions:
 - a. Status should be 200 OK.
 - b. Response should be a JSON object.
 - c. The users' array should not be empty.

- d. Each user in the results should have a firstName that includes the search keyword provided in the query parameter.

2. POST /users /Add

- Purpose: Creates a new user in the system with the provided details.
- Endpoint: POST {{baseUrl}}/users/add
- Body input from CSV file:

```
{
  "firstName": "{{UserFirstName}}",
  "lastName": "{{UserLastName}}",
  "age": {{UserAge}}
}
```
- Assertions:
 - a. Status should be 201 Created.
 - b. Response should be a JSON object.
 - c. Response should include an id for the newly created user.
 - d. The returned fields (firstName, lastName, age) should match the input values provided in the request body.

C. To ensure backend resilience, invalid requests were tested as shown below:

1. GET auth /me (missing token)

- Purpose: Attempts to retrieve the profile of a user without providing authentication.
- Endpoint: GET {{baseUrl}}/auth/me
- Assertions:
 - a. Status should be 401 Unauthorized.
 - b. Response body should include property (message → Access Token is required).

Note: Two failures occurred in this request because DummyJSON occasionally returns actual data instead of the expected error message, causing assertion mismatches.

2. GET /products /id (invalid)

- Purpose: Attempts to fetch a product with an invalid ID from DummyJSON
- Endpoint: GET {{baseUrl}}/products/{{invalidID}}
- Assertions:
 - a. Status should be 404 Not Found.
 - b. Response should be a JSON object containing a property (message → Product with id " not found)
 - c. Content-Type should be application/json.

6.4 Automation and Reporting

A. Postman Collection Runner (shown in Figure 4):

- Collections were executed using Postman's built-in Runner.
- CSV files were selected to enable data-driven execution.
- The iteration count was determined based on rows count in the CSV file.

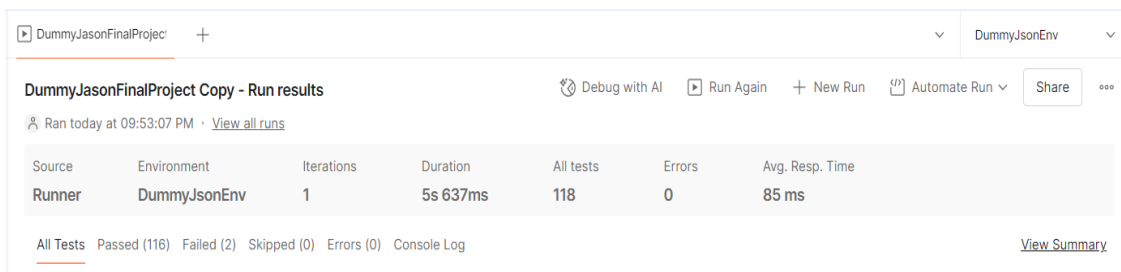


Figure 4: Postman Collection Runner Results

B. Newman CLI Execution:

- The same collection was executed via command line.
- Command used was (newman run DummyJson.json ^ -e Environment.json ^ -d UsersData.csv).
- Execution included: Collection file, Environment variables, CSV File
- Reporting Output:
 - XML, HTML Xtra reports, shown in Figure 5) were generated from the Newman run.
 - Reports provide:

- Total requests and assertions executed
- Detailed information for all requests
- Iteration-wise execution using CSV data
- Overall pass/fail status
- Total execution time

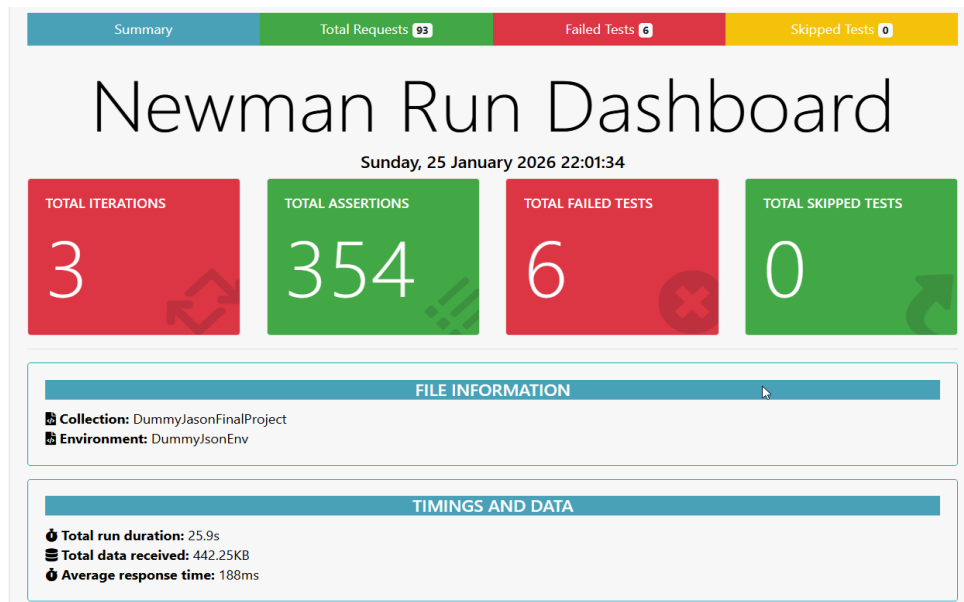


Figure 5: HTML Xtra Report

7. K6 Performance Testing

This part describes the performance testing strategy and results obtained using the K6 tool on extracted endpoints from the Postman collection. The goal was to evaluate backend API stability and responsiveness under different load conditions.

7.1 Test Plan and Load Profiles

The test targeted three endpoints in two stages:

- Endpoints: Get /carts (list), Get /products /id (single product), Get /users/ search.
- Smoke Test: Conducted on 3 VUs over a 30-second duration, to establish the baseline for the service functionalities under minimal load.
- Load Test: Conducted on 20 VUs over a 30-second duration to observe the behavior under concurrent user load.

7.2 Smoke Test Observation

It shows stability and responsiveness under minimal load along with the consistent low response time. The insignificant difference between average time and P(95) reflects a reliable performance with no bottlenecks, as provided in Figure 6:

```
C:\Users\HCC\OneDrive\OneDrive - Hashemite University\Desktop\FinalProjectDrafts\Final_K6>k6 run SmokeTest.js

      Grafana
    K6

execution: local
script: SmokeTest.js
output: -

scenarios: (100.00%) 1 scenario, 3 max VUs, 1m0s max duration (incl. graceful stop):
 * default: 3 looping VUs for 30s (gracefulStop: 30s)

■ THRESHOLDS

http_req_duration
✓ 'p(90)<250' p(90)=162.85ms
✓ 'p(95)<400' p(95)=301.5ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

■ TOTAL RESULTS

checks_total.....: 204      5.316943/s
checks_succeeded...: 100.00% 204 out of 204
checks_failed.....: 0.00%   0 out of 204

✓ 200 Status Code
✓ content-type = application/json
✓ carts property exists
✓ carts array length > 0
✓ response is json obj
✓ All carts have required keys
✓ Content-Type = application/json

HTTP
http_req_duration.....: avg=168.42ms min=147.43ms med=156.22ms max=304.33ms p(90)=162.85ms p(95)=301.5ms
{ expected_response:true }...: avg=168.42ms min=147.43ms med=156.22ms max=304.33ms p(90)=162.85ms p(95)=301.5ms
http_req_failed.....: 0.00% 0 out of 36
http_reqs.....: 36      0.938125/s

EXECUTION
iteration_duration.....: avg=9.55s min=9.47s med=9.49s max=9.74s p(90)=9.73s p(95)=9.74s
iterations.....: 12      0.312708/s
vus.....: 3 min=3 max=3
vus_max.....: 3 min=3 max=3

NETWORK
data_received.....: 469 kB 12 kB/s
data_sent.....: 9.6 kB 251 B/s
```

Figure 6: Smoke Test Results.

7.3 Load Test Observation

The System is functionally stable, yet the Maximum Response Time has spiked compared to the smoke test, which lead to higher concurrency. However, the Average Response Time and P(90) remained acceptable, the P(95) and Maximum Response Time sharp increase indicates connection overhead as a major factor in performance degradation, as illustrated in Figure 7.

```
execution: local
  script: LoadTest.js
  output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 20 looping VUs for 30s (gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(90)<550' p(90)=463.97ms
✓ 'p(95)<600' p(95)=523.86ms

http_req_failed
✓ 'rate<0.1' rate=0.00%

TOTAL RESULTS

checks_total.....: 1280      32.556703/s
checks_succeeded...: 100.00% 1280 out of 1280
checks_failed.....: 0.00%    0 out of 1280

✓ 200 Status Code
✓ Content-Type = application/json
✓ Response is an object
✓ carts property exists and is an array

✓ All carts have required keys
✓ Response is json object
✓ Valid required keys
✓ Valid key Types
✓ ReturnedID = RequestedID
✓ Returned Response is an Object
✓ Returned Response is Not Empty
✓ First Name has the Searched Word

HTTP
http_req_duration.....: avg=245.64ms min=151.14ms med=170.16ms max=665.45ms p(90)=463.97ms p(95)=523.86ms
{ expected_response:true }...: avg=245.64ms min=151.14ms med=170.16ms max=665.45ms p(90)=463.97ms p(95)=523.86ms
http_req_failed.....: 0.00%    0 out of 240
http_reqs.....: 240      6.104382/s

EXECUTION
iteration_duration.....: avg=9.77s   min=9.48s   med=9.65s   max=10.24s   p(90)=10.23s   p(95)=10.23s
iterations.....: 80      2.034794/s
vus.....: 18      min=18      max=20
vus_max.....: 20      min=20      max=20

NETWORK
data_received.....: 3.1 MB 80 kB/s
data_sent.....: 65 kB 1.6 kB/s

running (0m39.3s), 00/20 VUs, 80 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 30s
```

Figure 7: Load Test Results.

7.4 K6 Results Analysis

Analysis and Bottleneck Identification

The test achieved 100% success across both loads (shown in Table 11), yet the load test uncovered some performance bottlenecks:

- The Maximum Response Time climb by over 4.3s shows that the system is hitting a limit affecting a small percentage of requests severely.
- Under the load of 20 Vus, the system struggled to create a secure connection which

took around 1.6 seconds.

- c. There was an increase in http_req_blocked (avg 37.15 ms) suggests that requests are being queued waiting for an available slot.

Table 11: K6 Results

Metric	Smoke Test	Load Test	Comparison / Change
Users	3 VUs	20 VUs	+566.7%
Total Requests	489	501	+2.45%
Throughput	16.13 req/s	14.92 req/s	-7.5%, efficiency dropped slightly as users increased.
Minimum Response Time	149.45 ms	150.84 ms	+0.9%, fastest responses remained stable.
Avg. Response Time	174.26 ms	220.32 ms	+26.4%, it rose up under load
Max Response Time	592.96 ms	4324.98 ms	+629.4% Spike: outliers appeared under load.
P(90) Response Time	178.64 ms	202.33 ms	+13.3%
P(95) Response Time	205.65 ms	419.44 ms	+103.9%, it doubled for the slowest 5% of users
Checks (Pass/Fail)	2608 / 0	2672 / 0	100% Pass in both
Success Rate	100%	100%	No change
Failure Rate	0%	0%	No change

7.5 K6 How to Run

- a. Navigate to the installed directory, locate performance test required.
- b. Open terminal and execute the following commands:
 - To run the Smoke Test, k6 run SmokeTest.js
 - To run the Smoke Test, k6 run SmokeTest.js
- c. During execution, test results and execution logs are displayed in the console.

8. UI Automation

This section covers the automation of functional user journeys for both Web (Selenium) and Mobile (Appium) platforms.

8.1 Selenium, TestNG Web Automation

- Approach and Structural Design:
 - a. User roles cover Standard_User, Problem_User.
 - b. The automation suite is organized into separate packages for each user role, containing a devoted data class that contains shared inputs to be used as a base class for the test classes inherit from it, promoting code reusability and maintainability.
 - c. Tests are ordered based on prioritization and extensive assertions for each test case to ensure logical flow execution as expected.
 - d. The project is built using the Selenium WebDriver as the core automation framework, with TestNG serving as the primary test runner where is developed within the Eclipse IDE.
- How To Run Tests
 - d. For Execution, navigate to the project in Eclipse, locate the test class or test suite, and run it as a TestNG Test.
 - e. During execution, test results and execution logs are displayed in the console.
- Automated tests included a total of 11 test cases to cover the end-to-end workflow:
 - a. Test cases covered for Standard_User (shown in Figure 8) are (Login/ Logout, Add items/ Remove items from cart, View item descriptions, Item filtering and sorting, A valid/ Invalid checkout scenarios) which ensured a complete success. **Note:** Sad Checkout Scenario validates error handling when mandatory fields are missing.

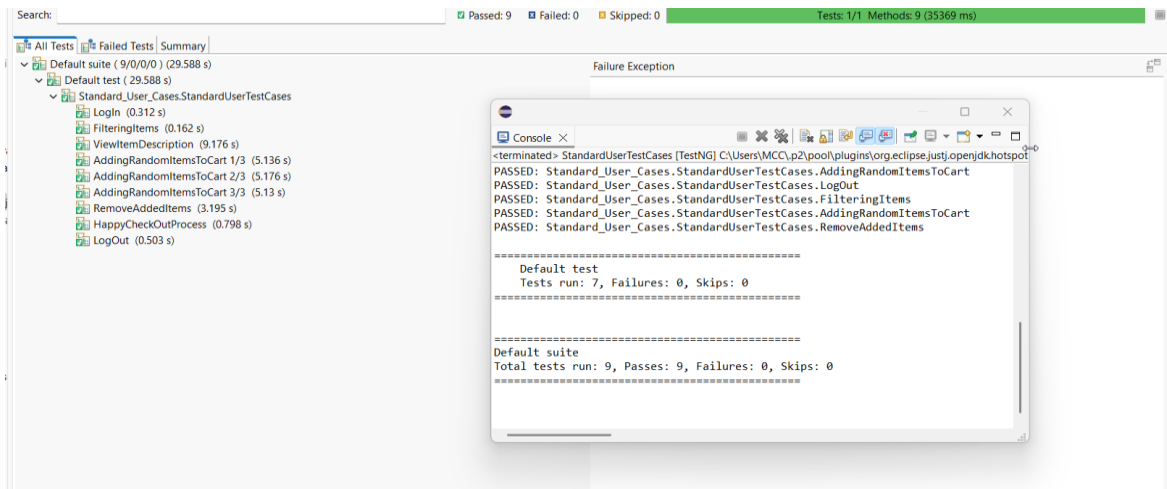


Figure 8: Standard_User Test Cases.

- b. Test cases covered for Problem_User (shown in Figure 9) are (Login, Add Items and Remove items from Cart) which identifies functional defects.

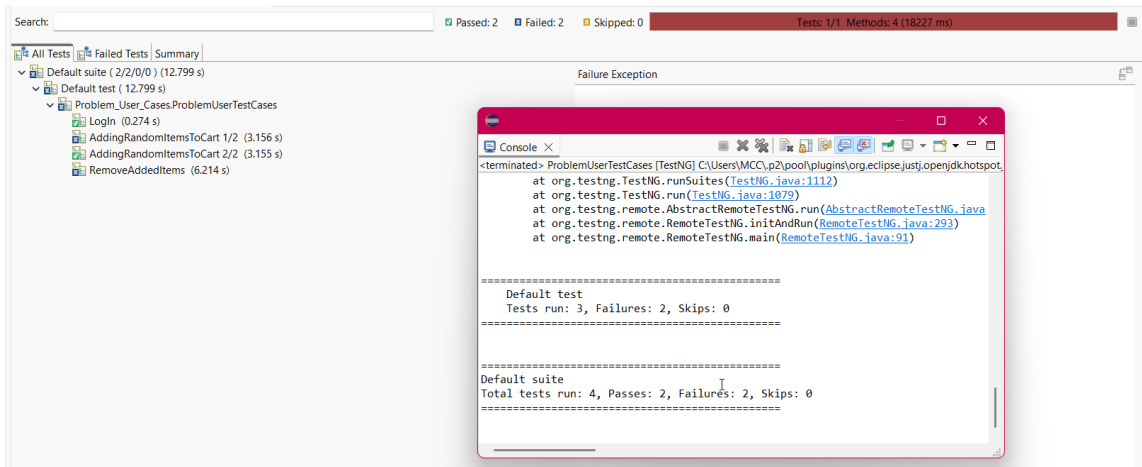


Figure 9: Problem_User Test Cases.

- Results and Reporting

Test output reports were generated by TestNG to provide a thorough view of the execution, capturing execution time, and confirming assertions were processed correctly. As shown in Figure 10.

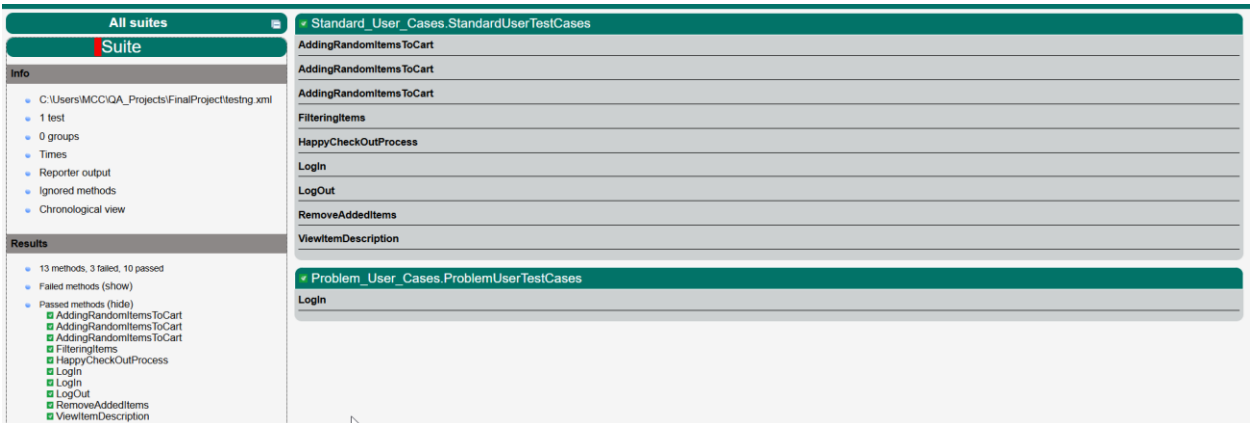


Figure 10: Test Output Reports.

8.2 Appium Mobile Automation

This section outlines the automated testing strategy for the HyperMax Android application using Appium. The focus is on ensuring a seamless native mobile experience, from initial onboarding to cart management.

- Approach and Structural Design

The strategy is designed to simulate a real user's journey.

- a. The flow script automates a complete shopping cycle: selecting a region, browsing specific categories (Frozen Food), managing quantities, and proceeding to check out.
- b. The automation interacts with native elements as quantity selectors and navigates to new screens ensuring the app's core UI logic remains intact.
- c. Every critical step is guarded by Chai assertions to verify that the performed action matches the intended action along with utilization of await to ensure the driver waits for elements to load before interacting.
- d. The project is built using WebdriverIO framework as the primary test runner, that executes the (.apk) file on a Pixel 5 Android Emulator.

- For Execution, navigate to the project directory and run the test script of (node HyperMaxTest.js)

Note: You can observe the real-time execution in the emulator and view the result log in the terminal.

- Sample Results

All automated steps, from country selection to checkout, were executed successfully.

- a. Onboarding: asserted that the "Confirm" button is displayed, as shown in Figure 11.

```
2026-02-09T09:28:07.566Z INFO webdriver: COMMAND findElement("id", "com.maf.hypermax:id/btn_confirm")
2026-02-09T09:28:07.568Z INFO webdriver: [POST] http://127.0.0.1:4723/session/c7b47774-fb09-4fbc-91a9-7cebde20e840/element
2026-02-09T09:28:07.568Z INFO webdriver: DATA {"using":"id","value":"com.maf.hypermax:id/btn_confirm"}
2026-02-09T09:28:07.631Z INFO webdriver: RESULT {
  'element-6066-11e4-a52e-4f735466cecf': '00000000-0000-019e-ffff-ffff000000b7',
  ELEMENT: '00000000-0000-019e-ffff-ffff000000b7'
}
2026-02-09T09:28:08.641Z INFO webdriver: COMMAND isElementDisplayed("00000000-0000-019e-ffff-ffff000000b7")
2026-02-09T09:28:08.642Z INFO webdriver: [GET] http://127.0.0.1:4723/session/c7b47774-fb09-4fbc-91a9-7cebde20e840/element/00000000-0000-019e-ffff-ffff000000b7/displayed
2026-02-09T09:28:08.674Z INFO webdriver: RESULT true
2026-02-09T09:28:08.676Z INFO webdriver: COMMAND elementClick("00000000-0000-019e-ffff-ffff000000b7")
2026-02-09T09:28:08.677Z INFO webdriver: [POST] http://127.0.0.1:4723/session/c7b47774-fb09-4fbc-91a9-7cebde20e840/element/00000000-0000-019e-ffff-ffff000000b7/click
```

Figure 11: "Confirm" Button Assertion Results.

- b. Quantity selection: Validated that selecting a specific quantity correctly updates the UI text to "2 Pieces", as shown in Figure 12.

```
2026-02-06T12:18:14.500Z INFO webdriver: COMMAND findElement("accessibility id", "SelectQuantity")
2026-02-06T12:18:14.500Z INFO webdriver: [POST] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
2026-02-06T12:18:14.501Z INFO webdriver: DATA {"using":"accessibility id","value":"SelectQuantity"}
2026-02-06T12:18:14.876Z INFO webdriver: RESULT {
  'element-6066-11e4-a52e-4f735466cecf': '00000000-0000-009f-ffff-ffff00000b46',
  ELEMENT: '00000000-0000-009f-ffff-ffff00000b46'
}
2026-02-06T12:18:16.890Z INFO webdriver: COMMAND getElementText("00000000-0000-009f-ffff-ffff00000b46")
2026-02-06T12:18:16.891Z INFO webdriver: [GET] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
t/00000000-0000-009f-ffff-ffff00000b46/text
2026-02-06T12:18:16.914Z INFO webdriver: RESULT 2 Pieces
2026-02-06T12:18:16.915Z INFO webdriver: COMMAND elementClick("00000000-0000-009f-ffff-ffff00000b46")
2026-02-06T12:18:16.915Z INFO webdriver: [POST] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
nt/00000000-0000-009f-ffff-ffff00000b46/click
```

Figure 12: Quantity Selection Assertion Results.

- c. Item Selection: Validated that selecting a specific item correctly updates the UI text to item name, as shown in Figure 13.

```
2026-02-06T12:18:02.397Z INFO webdriver: COMMAND isElementDisplayed("00000000-0000-009e-ffff-ffff0000076b")
2026-02-06T12:18:02.398Z INFO webdriver: [GET] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
t/00000000-0000-009e-ffff-ffff0000076b/displayed
2026-02-06T12:18:02.424Z INFO webdriver: RESULT true
2026-02-06T12:18:02.424Z INFO webdriver: COMMAND getElementText("00000000-0000-009e-ffff-ffff0000076b")
2026-02-06T12:18:02.425Z INFO webdriver: [GET] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
t/00000000-0000-009e-ffff-ffff0000076b/text
2026-02-06T12:18:02.442Z INFO webdriver: RESULT Alwadi Tender Chicken Breast 2 Kg
2026-02-06T12:18:03.444Z INFO webdriver: COMMAND findElement("accessibility id", "AddToCartButton")
2026-02-06T12:18:03.445Z INFO webdriver: [POST] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
nt
2026-02-06T12:18:03.445Z INFO webdriver: DATA {"using":"accessibility id","value":"AddToCartButton"}
2026-02-06T12:18:03.575Z INFO webdriver: RESULT {
  'element-6066-11e4-a52e-4f735466cecf': '00000000-0000-009e-ffff-ffff00000874',
  ELEMENT: '00000000-0000-009e-ffff-ffff00000874'
}
```

Figure 13: Item Selection Assertion Results.

- d. Input Validation Handling: Confirmed that clicking on "Continue" Button with empty fields properly triggers the validation logic to be displayed, as shown in Figure 14.

```
2026-02-06T12:18:29.386Z INFO webdriver: COMMAND elementClick("00000000-0000-00a2-3b9a-ce680000f2d")
2026-02-06T12:18:29.387Z INFO webdriver: [POST] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
nt/00000000-0000-00a2-3b9a-ce680000f2d/click
2026-02-06T12:18:29.387Z INFO webdriver: DATA {}
2026-02-06T12:18:30.630Z INFO webdriver: RESULT null
2026-02-06T12:18:31.637Z INFO webdriver: COMMAND findElement("-android uiautomator", "new UiSelector().text(\"Please ente
r an email address and continue.\")")
2026-02-06T12:18:31.637Z INFO webdriver: [POST] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
nt
2026-02-06T12:18:31.638Z INFO webdriver: DATA {"using":"-android uiautomator","value":"new UiSelector().text(\"Please en
ter an email address and continue.\")"}
2026-02-06T12:18:31.668Z INFO webdriver: RESULT {
  'element-6066-11e4-a52e-4f735466cecf': '00000000-0000-00a2-0000-047f00000f2d',
  ELEMENT: '00000000-0000-00a2-0000-047f00000f2d'
}
2026-02-06T12:18:33.678Z INFO webdriver: COMMAND isElementDisplayed("00000000-0000-00a2-0000-047f00000f2d")
2026-02-06T12:18:33.679Z INFO webdriver: [GET] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8/element
t/00000000-0000-00a2-0000-047f00000f2d/displayed
2026-02-06T12:18:33.710Z INFO webdriver: RESULT true
2026-02-06T12:18:36.716Z INFO webdriver: COMMAND deleteSession()
2026-02-06T12:18:36.717Z INFO webdriver: [DELETE] http://127.0.0.1:4723/session/43bb9583-5bdf-4b80-a744-45e4e61244f8
2026-02-06T12:18:37.351Z INFO webdriver: RESULT null
```

Figure 14: Input Validation Handling Assertion Results.

9. CI/CD Integration

CI/CD pipeline was implemented to automate test execution and ensure continuous validation of system quality.

9.1 Setup

- Git was employed git commands to manage, synchronize files with GitHub repository.
- A self-hosted GitHub Actions runner was configured on a Linux environment for CI/CD integration, enabling performance and API execution upon updates.

9.2 Pipeline Trigger

The pipeline is triggered only on the main branch, ensuring automated tests validate stable, production-ready code.

9.3 Workflow Automation

A workflow file (.github/workflows/deploy.yml) was created to automatically run Smoke Tests and API Test.

9.4 Test Execution & Reporting

Test execution and results are generated after updates, providing early detection of failures and ensuring that only stable builds progress through the pipeline.

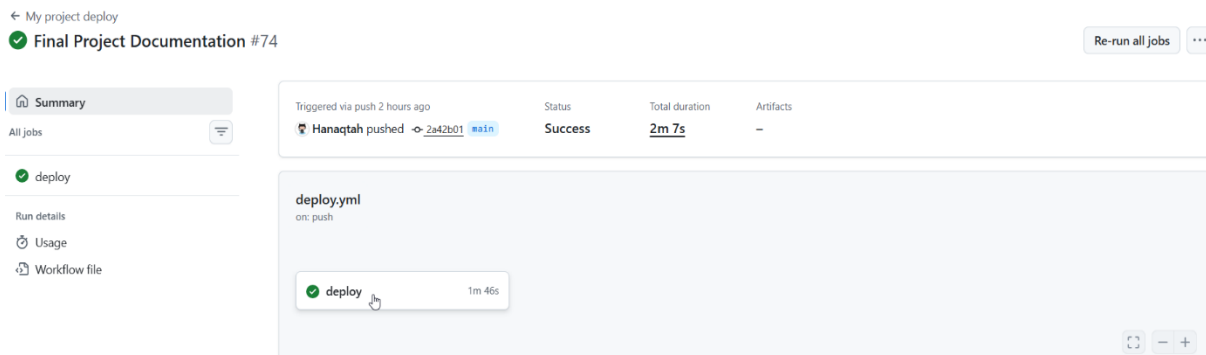


Figure 15: CI/CD deploy.yml File Run Result

10. Risks, Limitation, and Recommendation

This section outlines the technical constraints, risks encountered during the testing lifecycle, along with a strategic recommendation for future engineering iterations.

10.1 Risks

- Third-Party (DummyJSON) API Dependency, during testing, inconsistencies were observed where the API occasionally returned success data instead of expected error messages, e.g. (missing token).
- Performance Volatility happens when multiple users access the system at the same time, response times can spike dramatically (up to 6 times slower in our tests). If not optimized, this could cause slow responses, timeouts, or even crashes under real-world traffic.
- Inconsistent Performance Metrics, during performance testing, response times and execution metrics, thresholds vary even under similar test conditions causing false performance failures. This is likely due to the API server's limited capacity to handle load and poses a risk of misleading performance results.

10.2 Limitation

- Stateful Testing Constraints, stateful changes (like adding a user or product) have not persisted permanently in DummyJSON Api, causing limitation to perform long-term data persistence testing.
- Limited Device Medium, as UI testing was primarily conducted on a Pixel 5 Android Emulator only. Yet UI rendering may vary across different mobile versions.
- Element Interception, during automation, pop-ups, dialog boxes, or background overlays can block underlying elements, causing them to be un-interactable, thus tests may fail unless these elements are explicitly handled.
- UI Changes Across App Versions, updates employed on application can alter element IDs, layout structures, or the order of displayed elements. As a result, false failures appear for the previous reliance on outdated identifiers or positions.

10.3 Recommendation

- For performance: Implement Circuit Breakers, to prevent one slow endpoint from dragging down the entire architecture, implement a circuit breaker to stop the system from waiting and return a fallback response.
- For API, use a persistent test environment, where stateful changes (are saved permanently).
- For automation:
 - a. A cross-browser integration with a cloud testing platform such as BrowserStack to execute the automated cases across a variety of real devices or browser versions.the automated cases across a variety of real devices or browser versions.
 - b. Maintainable Test Scripts: Apply the Page Object Model (POM) to separate element locators from test logic, making scripts easier to maintain when the UI changes from test logic, making scripts easier to maintain when UI changes.

11. Appendices

Appendix 1: Project Repository on GitHub Link

<https://github.com/Hanaqtah/FinalProject-HTU>

Appendix 2: Test Cases Excel Sheet

[ProjectTestCases.xlsx](#)

Appendix 3: Bug Report Links

[Item9_BugReport.xlsx](#)

[Item12_BugReport.xlsx](#)

[Item16_BugReport.xlsx](#)

[Item14_BugReport.xlsx](#)

[Item18_BugReport.xlsx](#)

[Item24_BugReport.xlsx](#)

[Item25_BugReport.xlsx](#)

[Item30_BugReport.xlsx](#)

[Item20_BugReport.xlsx](#)